arXiv

License: arXiv.org perpetual non-exclusive license (https://info.arxiv.org/help/license/index.html#licenses-available)
arXiv:2610.26239v1 [cs.AI] 19 Oct 2026

CoRS: Fast Neural Flexible Job-Shop Scheduling via Composite Interval State Reuse

Sutong Jiang

Peng Zhao

Xuan Wu

Yan Wang

Jiaqi Wang

Rui Zhang\corresponding

Yuan Jiang\corresponding

You Zhou\corresponding

Di Wang

Abstract

Neural solvers for the Flexible Job-Shop Scheduling Problem (FJSP) typically refresh graph representations and update feasibility after every operation–machine decision. However, only feasibility must remain exact at every step, whereas graph encoding is substantially more expensive and can potentially be refreshed less frequently. We present CoRS, which decouples them: an interval-level graph representation is reused across multiple sequential decisions, while a stepwise feasibility state is updated after every action. This two-timescale design amortizes graph encoding without parallelizing action generation or relaxing scheduling constraints. CoRS further develops a feasibility-preserving composite-interval decoding scheme with dynamic constraint tracking and boundary state synchronization, which reduces representation refresh frequency while maintaining continuous feasible schedule construction. Furthermore, to alleviate representation staleness caused by interval-level state reuse, CoRS introduces enhanced static features as a compensation mechanism. By providing more comprehensive structural information of the scheduling problem, these features improve the model's perception of overall problem characteristics. Extensive experiments on synthetic and multiple public FJSP benchmarks demonstrate that CoRS significantly reduces inference time while maintaining competitive solution quality, achieving favorable speed–quality trade-offs under tight wall-clock budgets and low-latency scenarios.

[1]College of Computer Science and Technology, Jilin University, China

[2]College of Software, Jilin University, China

[3]College of Computing and Data Science, Nanyang Technological University, Singapore

[4]Joint NTU-UBC Research Center of Excellence in Active Living for the Elderly, Nanyang Technological University, Singapore

Introduction

With the advent of the Industry 4.0 era, the rapid development of information technologies such as the Internet of Things (IoT) (Li et al. 2015) and artificial intelligence (AI) (Mao et al. 2019) has accelerated the digitalization of manufacturing systems, making cloud manufacturing an important mechanism for modern manufacturing (Bo-Hu et al. 2010; Ren et al. 2017). As a core issue in manufacturing systems (Liu et al. 2015), scheduling problems require efficient decision-making based on cloud data. With the increasing scale and complexity of manufacturing systems, the response speed of scheduling decisions has become a critical challenge in cloud manufacturing environments (Kovacova and Lăzăroiu 2021; Zhang et al. 2025).

As a classical NP-hard Combinatorial Optimization Problem (COP) (Xie et al. 2019), the Flexible Job-Shop Scheduling Problem (FJSP) has become a modeling prototype for diverse applications, including medical resource scheduling (Abderrabi et al. 2021), semiconductor manufacturing (Ghaedy-Heidary et al. 2024), and aero-engine production (Zhou et al. 2019), owing to its enhanced flexibility (Chaudhry and Khan 2016). In practical manufacturing systems, scheduling systems are required to respond rapidly to changes in production states within limited time. Consequently, efficiently solving the FJSP has become a central challenge (Akbaripour et al. 2018; Xu et al. 2025).

Given the significance of FJSP, numerous methods have been proposed, including exact algorithms (Tutumlu and Saraç 2023), Priority Dispatching Rules (PDRs) (Branke et al. 2015), and metaheuristics (Xu et al. 2025). However, these methods suffer from limitations in solution quality, generalization, and computational efficiency (Xiao et al. 2025; Han and Yang 2021; Zhang et al. 2023), making them inadequate to meet the requirements of cloud manufacturing. Recently, deep learning has introduced new methods for solving FJSP. Neural combinatorial optimization methods show strong modeling and decision-making capabilities for scheduling problems (Wang et al. 2025; Liao et al. 2025; Smit et al. 2025). Most studies formulate the scheduling process as a Markov Decision Process (MDP), adopt autoregressive (AR) decision-making, and integrate reinforce-

ment learning for end-to-end optimization to obtain high-quality solutions (Yuan et al. 2024; Zhao and Deng 2024).

However, most existing neural FJSP solvers primarily focus on state representation learning or backbone architecture design (Xu et al. 2024), while implicitly assuming that the complete graph representation must be refreshed after every scheduling decision. In fact, the global state usually undergoes only limited changes between consecutive decision steps (see Appendix A for analysis). So existing methods perform complete feature encoding for highly similar states, resulting in substantial computation and significantly degrading solution efficiency.

Motivated by this observation, we question this assumption and explore a scheduling approach that reduces state updates while preserving dynamic feasibility. We propose the Composite Regressive (CR) mechanism, which redefines the state update process in neural FJSP solving. Unlike existing methods that implicitly assume the complete graph representation must be refreshed after every scheduling decision, CR introduces representation-refresh frequency as an underexplored design dimension and formulates a two-timescale state evolution mechanism. Specifically, CR factorizes the scheduling state into two coupled components with different update frequencies: an interval-level representation state and a stepwise feasibility state. The interval-level representation state is refreshed only at composite-interval

boundaries, while the stepwise feasibility state maintains dynamic constraints and ensures schedule feasibility by progressively updating the feasible action set.

Based on this mechanism, we further develop the **Co**mposite **R**egressive **S**cheduling (CoRS) model for solving FJSP. Unlike blockwise or parallel autoregressive methods, CoRS does not reduce the number of action-generation steps; it reduces the number of representation-refresh steps. Unlike naive representation caching, CoRS explicitly tracks the evolving feasibility state and periodically reconstructs the full scheduling state. By reusing interval-level graph representations across consecutive scheduling decisions, CoRS reduces the number of graph encoding operations from T to $\lceil T/X \rceil$, while maintaining scheduling feasibility through the progressive update of the stepwise feasibility state.

However, reducing the frequency of state refreshes may cause the interval-level representations to fail to timely capture certain dynamic changes, thereby affecting the quality of action ranking. To alleviate this issue, CoRS introduces enhanced static features as a compensation mechanism. By providing more comprehensive structural information of the scheduling problem, these features improve the model's perception of global scheduling characteristics and enhance the effectiveness of interval representations under low-frequency updates. Meanwhile, state synchro-

nization at interval boundaries further limits the accumulation of representation staleness, ensuring that its impact is mainly restricted to the ranking among feasible actions rather than compromising scheduling feasibility.

Specifically, we first propose a two-timescale state evolution mechanism that decouples the scheduling state into an interval-level representation state and a stepwise feasibility state. The interval-level representation state adopts composite intervals as the state refresh units rather than traditional decision steps, and is updated only at composite-interval boundaries. By reusing interval-level state representations across consecutive scheduling decisions, this mechanism reduces encoding of complete graph representations and alleviates the inference overhead caused by frequent state updates. Meanwhile, the stepwise feasibility state continuously tracks dynamic constraint changes, ensuring the feasibility of sequential schedule construction. Furthermore, we introduce enhanced static features as a compensation mechanism for interval-level state reuse. By providing more comprehensive structural information of the scheduling problem, these features improve the model's perception of overall problem characteristics and mitigate representation staleness caused by low-frequency state refreshing. Extensive experiments conducted against four state-of-the-art methods (Wang et al. 2023; Zhao et al. 2025; Song et al. 2022; Xiao et al. 2026) and four well-known PDRs demonstrate that CoRS can generate competitive solutions while significantly improving inference speed.

The main contributions of this study are summarized as follows:

- We expose representation-refresh frequency as an under-explored design axis in neural FJSP solving and formulate a two-timescale state evolution that decouples interval-level graph representations from stepwise feasibility updates.
- We develop a feasibility-preserving composite-interval decoding scheme with dynamic constraint tracking and boundary state synchronization. It reduces graph-encoder invocations from T to $\lceil T/X \rceil$ while retaining sequential and feasible schedule construction.
- We provide same-backbone and interaction studies that isolate the effect of representation reuse and show that enhanced static features increasingly compensate for representation staleness as the interval length grows. Fixed-budget comparisons further demonstrate the practical quality–latency trade-off.

Related Work

Traditional methods for solving FJSP mainly include exact methods, PDRs, and metaheuristic algorithms. Exact methods construct FJSP as a mathematical model and explore the entire solu-

tion space to find the optimal solution, such as branch and bound (Brucker et al. 1994), integer programming (Meng et al. 2020), and constraint programming (Müller et al. 2022). PDRs directly guide the scheduling decision process using manually designed priority rules, such as the Shortest Processing Time (SPT) rule (Chen and Matis 2013) and the Most WorK Remaining (MWKR) rule (Branke et al. 2015; Sels et al. 2012). Metaheuristic algorithms iteratively explore the solution space through carefully designed search strategies to obtain high-quality solutions, such as genetic algorithms (Gao et al. 2019) and particle swarm optimization (Nouiri et al. 2018; Zhang et al. 2009). However, exact methods struggle to handle large-scale instances (Demir and İşleyen 2013; Zhang et al. 2023). PDRs based on fixed heuristics fail to capture complex scheduling patterns, resulting in low solution quality (Chen and Matis 2013). Metaheuristic algorithms require extensive iterative searches, incurring high computational costs (Caldeira and Gnanavelbabu 2019). Furthermore, traditional methods typically solve each unseen instance from scratch, which limits their generalization ability.

In recent years, deep learning-based neural solvers have received increasing attention for solving FJSP. To enhance the expressive power for complex scheduling problems, existing neural FJSP solvers often incorporate graph neural networks (Zeng et al. 2022) or Transformer-based architectures (

Xu et al. 2024) to model operation–machine interactions and precedence dependencies.Very recently,ReSched (Xiao et al. 2026) explored simplified state representations for Transformer-based FJSP solving and reported promising results. Parallel autoregressive methods, such as PARCO (Berto et al. 2026) and MACSIM (Luttmann and Xie 2025), alleviate the sequential dependency of conventional autoregressive approaches by predicting multiple actions within a single decoding stage. These methods mainly accelerate scheduling from the perspective of action generation. However, existing solvers typically still require the complete graph-state representation to be refreshed after each decision. Such frequent representation updates and environment interactions introduce substantial computational overhead, significantly increasing inference time. As the scale of FJSP grows, inference time increases linearly with the number of time steps (Demir and İşleyen 2013), thereby exposing significant efficiency bottlenecks in large-scale or real-time scheduling scenarios.

Recently, some studies have explored fast solving approaches based on fixed state representations, such as non-autoregressive (NAR) methods. NAR methods have been widely applied to the traveling salesman problem (Dat et al. 2025) and the vehicle routing problem (Xiao et al. 2024b). However, NAR models generally suffer from degraded solution quality (Joshi et al. 2019). This limitation mainly arises because NAR models make decisions based on static state representations, making it difficult to

capture the dynamic changes in scheduling states caused by sequential decisions. As a result, scheduling solutions can be generated efficiently, but they often deviate from high-quality solutions (Xiao et al. 2024a). In FJSP, the relationships among nodes are highly complex and the scheduling state changes continuously with decisions, making it difficult to complete all decisions under a fixed state representation. Consequently, to our knowledge, fully non-autoregressive neural construction methods that directly generate complete feasible FJSP schedules remain underexplored; existing fast-learning or two-stage approaches either decompose the problem, rely on solver-assisted post-processing, or still construct schedules sequentially.

Overall, existing neural FJSP solvers mainly focus on state representation learning and network architecture design, while representation-refresh frequency, as an important design dimension affecting inference efficiency, remains largely unexplored. Although fixed state representation methods can reduce update overhead, they struggle to adapt to dynamically changing feasibility constraints in FJSP. To address this problem, we propose the CoRS, which decouples interval-level representation reuse and stepwise feasibility updates through a two-timescale state evolution design, reducing state refresh overhead while maintaining dynamic feasibility during scheduling. This makes CoRS complementary to recent representation- and backbone-level advances in neural FJSP solving.

Preliminaries

This section introduces the CR mechanism and presents the formulation of the FJSP.

Composite regressive mechanism. The CR mechanism adopts composite intervals as the update units for graph-state representations, rather than re-encoding the complete graph representation after each scheduling decision. Within each composite interval, CoRS reuses the interval-level graph-state representation and only updates the stepwise feasibility state to track dynamically changing constraints during scheduling. At composite-interval boundaries, the interval-level graph-state representation is synchronously reconstructed, reducing redundant graph encoding overhead and preventing outdated information from accumulating due to infrequent representation updates. Through this two-timescale state evolution mechanism, CoRS reduces graph-state refresh frequency and inference overhead while ensuring that action selection satisfies dynamic constraints through stepwise feasibility updates, thereby achieving a better efficiency–quality trade-off.

Under the CR mechanism, the entire FJSP scheduling process is divided into multiple consecutive, non-overlapping composite intervals B. B^{τ} denotes the τ-th composite interval, which contains X time steps and is defined by:

$$B^{\tau} = \{t | (\tau - 1)X + 1 \leq t \leq \min(\tau X, t_{\text{final}})\} \tag{1}$$

where $\tau = 1, 2, ..., \tau_{\max}$, t_{final} denotes the final time step of the entire scheduling process, and $\tau_{\max}$ represents the total number of composite intervals.

Flexible job-shop scheduling problem. The FJSP is defined as follows. Given a set of n jobs $J = \{J_1, J_2, ..., J_n\}$ and a set of m machines $M = \{M_1, M_2, ..., M_m\}$, each job J_i consists of n_i operations $O_i = \{O_{i1}, O_{i2}, ..., O_{in_i}\}$. Each operation O_{ij} has a compatible machine set $M_{ij} \subseteq M$ and can be processed on any machine $M_k \in M_{ij}$ with processing time p_{ijk}, where the completion time is denoted by C_{ij}. The objective is to minimize the maximum completion time $C_{\max} = \max\{C_{in_i} | \, i = 1, 2, ..., n\}$. The scheduling process must satisfy the following constraints: the operations of each job must follow their precedence constraints, and each machine can process at most one operation at a time with non-preemptive processing.

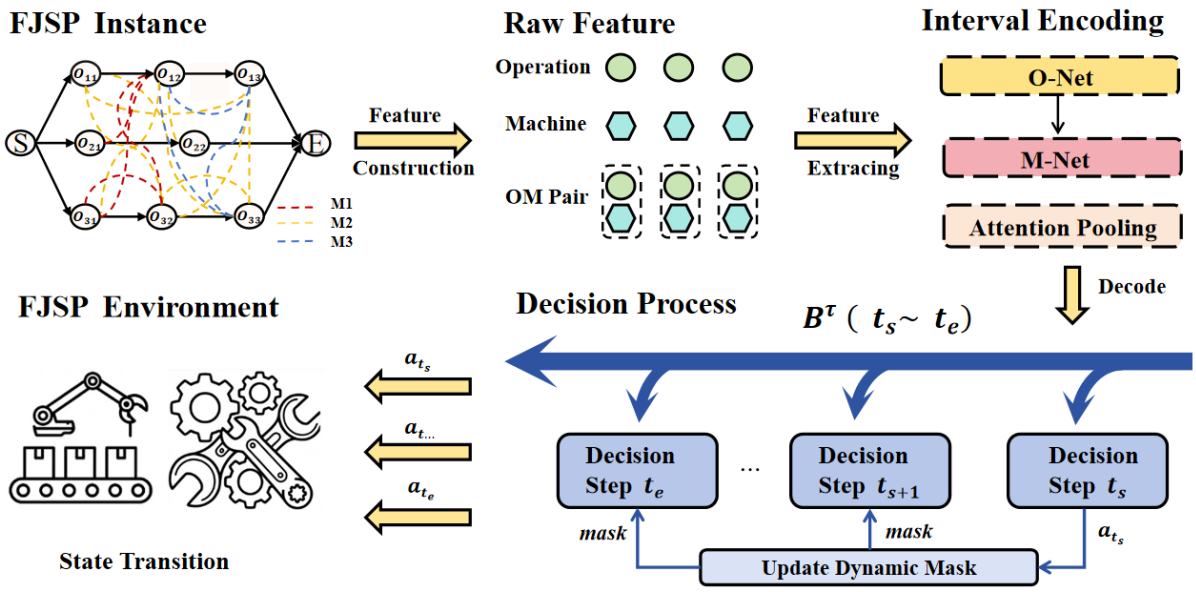


Figure 1: The architecture of CoRS.

Methodology

This section introduces the CoRS model, the scheduling model based on the CR mechanism, which achieves efficient inference for the FJSP. Figure 1 illustrates the workflow of CoRS. Through a specially designed interval encoding network and decoding algorithm, CoRS generates high-quality scheduling policies while significantly reducing inference time. In the following, we sequentially present the MDP, the feature encoding scheme, the interval encoding network, the composite-interval decoding algorithm, and the reinforcement learning algorithm.

MDP Formulation

Unlike the MDP used in traditional reinforcement learning, under the CR mechanism, CoRS formulates a tailored MDP for FJSP by considering the characteristics of reusing graph representations within each composite interval and updating the graph representations across intervals. The MDP is defined as follows.

State. Within the composite interval B^{τ}, CoRS adopts a two-timescale state evolution mechanism, which decomposes the scheduling state into an interval-level representation state s^{τ} and a stepwise feasibility state s_t^{feas} with different update frequencies. Specifically, s^{τ} is defined as $s^{\tau} = \{h_{ij}, h_k, h_{(ij,k)}\}$, where h_{ij}, h_k, and $h_{(ij,k)}$ denote the features of operation O_{ij}, machine M_k, and operation–machine (O_{ij}, M_k) pair, respectively. Meanwhile, s_t^{feas} records the step-wise updated feasibility information, including candidate operations, operation–machine feasibility masks, and machine release times. Different from the s^{τ}, s_t^{feas} is only used to determine the feasible action space and does not participate in state feature encoding or reconstruction within the composite interval.

Action. For any time step t, the action space A_t consists of all feasible OM pairs at the current scheduling state. The model selects one OM pair from A_t as the action a_t.

State Transition. Under the CR mechanism, state transitions follow a two-timescale update process. For the interval-level representation state, the state is synchronized only at composite-interval boundaries. The next state is defined as $s^{\tau+1} = f_{\text{recon}}\left(s^{\tau}, \{a_t\}_{t \in B^{\tau}}\right)$, where $f_{\text{recon}}(\cdot)$ denotes the vectorized state reconstruction function based on the action sequence within the composite interval. Meanwhile, s_t^{feas} is updated after each operation–machine assignment to track dynamic constraint changes and maintain the feasible action space at each decision step.

Reward. For any operation O_{ij} at time step t, its estimated completion time is denoted as $C_{\text{LB}}(O_{ij}, t)$. For scheduled operations, $C_{\text{LB}}(O_{ij}, t)$ equals the actual completion time C_{ij}. For unscheduled operations, the estimated completion time of all initial operations is initialized as $C_{\text{LB}}(O_{i0}) = 0$. The estimated completion time is recursively computed based on the completion time of the predecessor operation and the minimum processing time among candidate machines, given by $C_{\text{LB}}(O_{ij}, t) = C_{\text{LB}}(O_{i,j-1}, t) + \min_{k \in M_{ij}} p_{ijk}$. Accordingly, an estimate of the overall makespan at time step t is given by $C_{\text{LB}}(t) = \max_{1 \le i \le n,\ 1 \le j \le n_i} C_{\text{LB}}(O_{ij}, t)$. During the scheduling process, the estimated completion times are updated as the agent executes actions. The reward is defined as the difference between the estimated makespans at two consecutive time steps, given by $r_t = C_{\text{LB}}(t) - C_{\text{LB}}(t+1)$.

Feature Encoding

Unlike conventional methods that re-encode state representations at each decision step, CoRS reduces redundant encoding operations by reusing interval-level representation state within composite intervals. Therefore, the state representation obtained from a single encoding process needs to incorporate more instance-related static information to support effective decision making over subsequent multiple scheduling steps. Based on this consideration, we propose an enhanced static feature encoding scheme. Specifically, given an FJSP instance as input, we encode it into operation features $h_{ij} \in \mathbb{R}^{12}$, machine features $h_k \in \mathbb{R}^{10}$, and OM pair features $h_{(ij,k)} \in \mathbb{R}^{8}$. We enhance the static features by incorporating information such as the average processing time of successor operations and the total number of fastest operations for each machine, as detailed in Appendix B.

Interval Encoding Network

Under the CR mechanism, the CoRS performs unified encoding of interval-level representation state features only at the initial step of each composite interval, reuses the resulting feature representations throughout subsequent consecutive decision steps, and re-executes global encoding after composite interval completion. As illustrated in Figure 2, the CoRS model employs a graph attention framework composed of an Operation Encoding Subnetwork (O-Net) and a Machine Encoding Subnetwork (M-

Net) to model operation nodes and machine nodes, respectively.

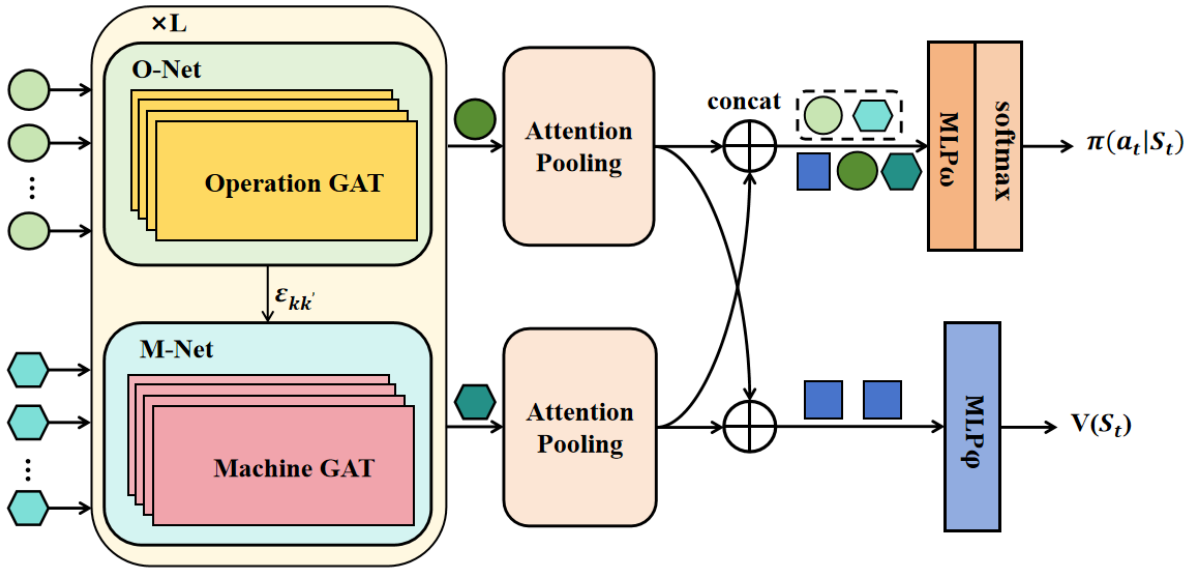


Figure 2: The network architecture of the CoRS.

Operation Encoding Subnetwork

O-Net models the operation states within the composite interval. In the $l \in L$-th multi-head graph attention layer, h_{ij} is first linearly projected as $z_{ij} = Wh_{ij}$, where $z_{ij} \in \mathbb{R}^{Hd_h}$ and $W \in \{W', W_1W_2\}$. H denotes the number of attention heads and d_h denotes the output dimension of each head. The selection of the projection matrix W depends on the layer index. For the first layer, a full shared matrix $W' \in \mathbb{R}^{Hd_h^{(1)} \times d_{in}^o}$ is used to map the input features into the

representation space of all attention heads. For intermediate layers, a low-rank factorization $W_1 W_2$ is adopted, where $W_1 \in \mathbb{R}^{Hd_h^{(l)} \times d_{mid}}$ and $W_2 \in \mathbb{R}^{d_{mid} \times Hd_h^{(l-1)}}$ with $d_{mid} < H \times d_h^{(l-1)}$, which reduces the number of parameters while preserving the projection dimension.

The vector z_{ij} is split into H sub-vectors as $z_{ij} = [z_{ij}^{(1)} \| \cdots \| z_{ij}^{(H)}]$, where $[a \| b]$ denotes vector concatenation and $z_{ij}^{(h)} \in \mathbb{R}^{d_h}$. To mitigate the reduced head diversity caused by the shared projection, a learnable scaling parameter β^h is applied to obtain $\tilde{z}_{ij}^h = \beta^h \odot z_{ij}^{(h)}$. In the h-th attention head, the attention coefficient between operation O_{ij} and its neighboring operation O_{ip} is computed as:

$$e_{ijp} = \text{LeakyReLU}\left(a^\top [\tilde{z}_{ij}^h \| \tilde{z}_{ip}^h]\right), \quad p \in \{j-1, j, j+1\}$$

where $a \in \mathbb{R}^{2d_h}$. The coefficients e_{ijp} are normalized by a softmax function to obtain the attention weights α_{ijp}. The output feature of this head is computed as:

$$\tilde{h}_{ij}^h = \sum_{p \in N_{ij}} \alpha_{ijp} \tilde{z}_{ip}^h \tag{3}$$

where $N_{ij} = \{j-1, j, j+1\}$. After obtaining the operation features from all attention heads, the intermediate layers concatenate the multi-head outputs, while the final layer adopts mean aggregation. The unified operation node representation is computed as $h_{ij}^{'(\tau)} = \text{ELU}\left(\frac{1}{H}\sum_{h=1}^{H} \tilde{h}_{ij}^h\right)$.

Machine Encoding Subnetwork

M-Net encodes machine node features to generate representations of machine states. In addition, M-Net introduces competition edge features. Specifically, $I_{kk'}$ denotes the set of unscheduled operations jointly competed by machines M_k and $M_{k'}$. If $I_{kk'} \neq \varnothing$, the two machines are considered to have a competition relationship. Accordingly, the set of machines competing with M_k is defined as $N_k = \{M_{k'} | I_{kk'} \neq \varnothing\}$. To quantify the competition intensity, we define the competition strength between them as the aggregation of the features of candidate operations in the candidate set J_c, given by $\epsilon_{kk'} = \sum_{O_{ij} \in I_{kk'} \cap J_c} h_{ij}$.

At the feature projection stage, we follow the same design as O-Net. $\tilde{z}_k$ and $\tilde{z}_{k'}$ denote the projected representations of M_k and $M_{k'}$, and $\tilde{z}_{kk'}$ denotes the projected representation of their competition edge. The attention coefficient is computed as $u_{kk'} = \mathrm{LeakyReLU}(b_k^\top \tilde{z}_k^h + b_{k'}^\top \tilde{z}_{k'}^h + b_{kk'}^\top \tilde{z}_{kk'}^h)$, where $b_k, b_{k'}, b_{kk'} \in \mathbb{R}^{d_h}$. $u_{kk'}$ is normalized by softmax to obtain the attention weight $\alpha_{kk'}$, which is used to aggregate machine features and produce the single-head output $\tilde{h}_k$. After obtaining the outputs of all heads, M-Net adopts the same multi-head fusion strategy as O-Net to generate the final machine embedding $h_k^{'(\tau)}$.

Attention Pooling

After encoding operation and machine nodes with O-Net and M-Net, CoRS constructs a global state representation for B^{τ} via attention-based global pooling.

Both machine and operation nodes are aggregated using the same attention mechanism. Taking machines as an example, a shared linear scoring function $g(\cdot)$ is used to measure the importance of M_k in the global representation, defined as $g_k = g(h_k^{'(\tau)})$, where $g_k \in \mathbb{R}$. The attention scores are normalized by softmax to obtain weights γ_k. The global machine representation is given by:

$$h_{\text{global}}^{(m,\tau)} = \sum_{k=1}^{M} \gamma_k h_k^{'(\tau)} \tag{4}$$

Similar to machines, operation nodes are aggregated by importance using the same attention pooling, giving $h_{\text{global}}^{(o,\tau)} = \sum_{i=1}^{n} \sum_{j=1}^{n_i} \gamma_{ij} h_{ij}^{'(\tau)}$, where γ_{ij} is the attention weight of O_{ij}. The overall state of B^{τ} is constructed by concatenating the machine and operation global representations as $h_{\text{global}}^{\tau} = [h_{\text{global}}^{(m,\tau)} \| \, h_{\text{global}}^{(o,\tau)}]$, providing a global representation for decision-making and value evaluation.

Decoding Algorithm

To reduce the inference overhead caused by frequent environment interactions and state updates, we propose a composite-

interval decoding algorithm that minimizes these updates. At time step t, the feasible action set is defined as:

$$A_t = f_{\text{mask}}\left(s_{t-1}^{\text{feas}}, a_{t-1}\right) \tag{5}$$

The function $f_{\text{mask}}(\cdot)$ updates the mask to restrict feasible actions without modifying state features. Specifically, $f_{\text{mask}}(\cdot)$ first updates the release times of the relevant machines according to the executed actions, and updates the current candidate operations of the scheduled jobs along with their machine compatibility relations. Then, based on the updated machine states, it calculates the earliest start time of each candidate operation–machine pair. Finally, operation–machine pairs that violate scheduling constraints are marked as infeasible, and the corresponding actions are masked out through the dynamic feasibility mask, thereby constructing the feasible action set for the current decision step. After completing B^{τ}, the next input scheduling state representation $s^{(\tau+1)}$ is reconstructed once in a vectorized manner by $f_{\text{recon}}(\cdot)$. Only the relative completion time of operations requires sequential updates due to strict order dependency, while other features are updated in batch by computing the effects of all actions in the composite interval. This produces the accurate state $s^{(\tau+1)}$ for subsequent scheduling decisions.

Training Algorithm

CoRS is trained under an Actor–Critic framework (

Zhao et al. 2025). Concatenated interval features are fed into the policy network, and the action distribution at state s_t is defined as:

$$\pi_\omega(a_t|\, s_t) = \sigma\Big(\mathrm{MLP}_\omega[h_k^{'(\tau)} \| \, h_{ij}^{'(\tau)} \| \, h_{(ij,k)}^{(\tau)} \| \, h_{\mathrm{global}}^{\tau}]\Big) \tag{6}$$

where σ denotes softmax. The value network evaluates the current state and guides policy optimization, taking $h_{\mathrm{global}}^{\tau}$ as input. The state value is defined as $v_\phi(s_t) = \mathrm{MLP}_\phi(h_{\mathrm{global}}^{\tau})$. The policy is optimized using Proximal Policy Optimization with clipping (PPO-Clip) (Schulman et al. 2017), where the clipping mechanism constrains the magnitude of policy updates to improve training stability. Generalized Advantage Estimation (GAE) (Schulman et al. 2015) is employed to compute the advantage function for improved optimization performance. The complete training procedure of CoRS is shown in Appendix C.

Experiments

This section describes the experimental setup, reports results on synthetic and public datasets, and analyzes the impact of the composite interval time step X as well as the ablation results.

Experimental Settings

Baselines. The four selected PDRs are described as follows:

FIFO schedules the OM pair that becomes available first; MOPNR schedules the OM pair whose operation has the largest number of remaining successor operations; SPT schedules the OM pair with the minimum processing time; MWKR schedules the OM pair with the maximum remaining average processing time. The three selected baselines are described as follows: HGNN (Song et al. 2022) models jobs and machines with a heterogeneous graph neural network to capture the relationships between operations and machines; DANIEL (Wang et al. 2023) adopts a reinforcement learning framework with a dual-attention network to improve solution quality; DOAGNN (Zhao et al. 2025) decouples the disjunctive graph into a dual graph structure and applies a dual operation aggregation graph neural network to represent operation states and their dependencies;ReSched (Xiao et al. 2026) proposes a Transformer-based simplified state representation framework, improving solution quality.

Datasets. We use $n \times m$ to denote an FJSP instance with n jobs and m machines. The model is trained on randomly generated instances of sizes 10×5, 20×5, 15×10, and 20×10, and evaluated on two synthetic datasets (SD1, SD2) and four public datasets (Hurink, Barnes, Brandimarte, and Dauzere). Detailed dataset descriptions are provided in Appendix D. Implementation details are in Appendix E.

Experiment Results and Analysis

Performance Comparison

Table 6 (Appendix F) and Table 7 (Appendix G) report the performance of CoRS and baseline methods on the synthetic and public datasets. LB denotes the known optimal solution for public datasets, while OR-Tools denotes results obtained by the Google OR-Tools exact solver on synthetic datasets with a time limit of 1800 s. Metrics include makespan ($C_{\max}$), relative gap to LB or OR-Tools (Gap), and average solving time per instance (Time/s). G and S denote the greedy and sampling strategies, respectively, where the sampling strategy performs 100 samples by default.

On the synthetic datasets, CoRS achieves significantly faster inference than all AR baselines. When $X = 2$, the speed improves by 42.94%–83.14%; when $X = 5$, the improvement further increases to 154.46%–238.76%. On the public benchmark datasets, CoRS achieves a 160.14%–489.86% speedup at $X = 5$. This acceleration mainly benefits from the CR mechanism, which reuses state encoding in a NAR manner within composite intervals, thereby substantially reducing state encoding operations.

While maintaining efficient inference, CoRS still produces high-quality solutions. When $X = 5$, the average gap to the best model on SD1 is only 4.04%, which further reduces to 0.71% on the public benchmarks. Compared with PDRs, CoRS also achieves

an 18.27%–41.18% improvement with comparable solving time. These results mainly benefit from the enhanced static features. Although increasing X reduces the model's perception of dynamic information and degrades solution quality, the enhanced static features can still effectively represent the global state, thereby maintaining stable performance.

Generalization Ability

We train the model on 20×10 instances and evaluate its generalization on 30×10, 40×10, 60×10, and 100×30 instances, with results reported in Table 5 (Appendix H). When $X = 3$, CoRS achieves high-quality solutions across all scales while maintaining the lowest inference time on SD1. On the 100×30 instances, CoRS achieves a 8.96% improvement in solution quality over AR models while maintaining lower inference time. On SD2, CoRS improves solution quality by 64.34% over PDRs while achieving a 129.85% inference speedup. These results demonstrate the strong generalization capability of CoRS. This is mainly due to the stable composite-interval state modeling of the interval encoding network and the effective constraints of the composite-interval decoding algorithm, allowing the model to maintain high inference efficiency and solution quality as the scale increases.

Time-Constrained Performance Evaluation

We evaluate solution quality under fixed average wall-clock bud-

gets on the 15×10 instances from SD1. For each method and budget, we report the best Cmax obtained by completed rollouts or sampling settings whose average wall-clock solving time per instance does not exceed the budget. As shown in Table 1, CoRS achieves competitive solution quality under tight time budgets. This advantage mainly stems from the CR mechanism, which reduces repeated state encoding and enables efficient schedule generation within limited inference time.

Table 1: Performance under Fixed Time Budgets

Time	DANIEL	RESHED	HGNN	DOAGNN	CoRS(X=2)
0.30	–	–	–	–	–
0.50	–	–	–	–	**160.36**
0.80	–	–	174.47	171.61	160.36
1.00	162.10	160.10	174.47	171.61	**157.41**
1.50	160.05	160.06	170.96	168.05	**156.67**
2.00	159.86	157.16	170.31	166.62	**155.90**
4.00	157.38	156.88	166.63	164.05	**154.63**

Impact of Composite Interval Steps (X)

We analyze the effect of X on model performance using 10×5

instances from SD1, normalizing all metrics relative to the AR case ($X = 1$). As shown in Figure 4 (Appendix J), increasing X substantially reduces inference time, which is only 22.1%–23% of AR at $X = 10$, while solution quality remains stable, varying by 3.3%–7.6%. This is primarily attributed to the proposed CR mechanism, which reduces inference time by decreasing state encoding, while the combination of enhanced static features with dynamic features maintains stable performance at larger time steps. In practical applications, the selection of X should be adjusted according to the trade-off between inference efficiency and solution quality. For scenarios with stringent real-time requirements, a larger X can be adopted to reduce the frequency of state encoding and decrease inference latency. In contrast, for scenarios where solution quality is prioritized, a smaller X should be selected to increase the frequency of state updates and maintain high-quality scheduling solutions.

Ablation Studies

As shown in Table 4 (Appendix I), we conduct a series of ablation experiments to evaluate the contributions of enhanced static features, the interval encoding method, and the composite-interval decoding strategy. The results show that removing enhanced static features (i.e., w/o En-fea) weakens node state representation, resulting in reduced solution quality. Furthermore, we analyze the interaction between X and enhanced features, as shown in Table 2. The experimental results demonstrate that the performance gains brought by enhanced static features gradually in-

crease with larger X. This indicates that enhanced static features provide richer and more persistent structural information within composite intervals, thereby enabling stable decision-making. Removing the interval encoding method (w/o Int-encoder) restores the step-by-step encoding strategy, resulting in only a slight improvement in average solution quality of 2.13% while substantially increasing solving time by 223.81%. Removing the composite-interval decoding strategy (i.e., w/o Co-decoder) causes more frequent state–environment interactions, thereby reducing inference efficiency.

Furthermore, since CoRS is built upon the DANIEL framework, we conduct a progressive comparison from the original DANIEL to the complete CoRS model. As shown in Table 3. Compared with DANIEL, introducing interval state reuse alone significantly reduces inference time, demonstrating that the CR mechanism effectively accelerates inference through interval-level state reuse. After further incorporating enhanced static features, CoRS improves solution quality, indicating that enhanced static features effectively compensate for the performance degradation caused by the reduced state update frequency during interval state reuse.

Table 2: Interaction between X and enhanced static features

Method		X			
		2	**3**	**4**	**5**
CoRS	G	107.91	108.26	108.93	109.35
	S	101.94	102.18	101.94	102.62
w/o En-fea	G	108.89	110.25	111.66	113.56
	S	102.34	103.45	104.55	109.63

Table 3: Progressive analysis from DANIEL to CoRS

Rdata	DANIEL		DANIEL+CR		CoRS	
	Cmax	**Time**	**Cmax**	**Time**	**Cmax**	**Time**
G	1031.95	0.912	1044.98	0.307	1032.45	0.213
S	978.33	3.25	991.05	1.118	982.23	0.895

Conclusion

This paper proposes the Composite Regressive (CR) mechanism and develops the CoRS scheduling model based on this mechanism for solving FJSP. CoRS adopts a two-timescale state evolution mechanism that decouples the interval-level represen-

tation state from the stepwise feasibility state. Furthermore, CoRS designs a feasibility-preserving composite-interval decoding scheme , reducing state refresh overhead while maintaining sequential and feasible schedule construction. In addition, enhanced static features are introduced to alleviate representation staleness caused by interval-level state reuse. Extensive experiments demonstrate that CoRS significantly reduces inference time while maintaining competitive solution quality, achieving superior speed–quality trade-offs under tight time budgets and low-latency scenarios. A current limitation is that the composite interval length is manually selected. The current method assumes a static deterministic FJSP and an exact stepwise feasibility tracker. Moreover, stale representations may still affect the ranking among feasible actions, and the achievable speedup depends on graph encoding being a major component of inference cost. Future work will study adaptive interval selection and extend composite interval state reuse to stronger Transformer-based FJSP backbones for improved speed–quality trade-offs across instance scales and distributions.

References

Abderrabi et al. (2021) F. Abderrabi, M. Godichaud, A. Yalaoui, F. Yalaoui, L. Amodeo, A. Qerimi, and E. Thivet
Flexible job shop scheduling

problem with sequence dependent setup time and job splitting: hospital catering case study.
Applied Sciences **11** (4), pp. 1504.
Cited by: Introduction.

Akbaripour et al. (2018)

H. Akbaripour, M. Houshmand, T. Van Woensel, and N. Mutlu
Cloud manufacturing service selection optimization and scheduling with transportation considerations: mixed-integer programming models.
The International Journal of Advanced Manufacturing Technology **95** (1), pp. 43–70.
Cited by: Introduction.

Berto et al. (2026)

F. Berto, C. Hua, L. Luttmann, J. Son, J. Park, K. Ahn, C. Kwon, L. Xie, and J. Park
PARCO: parallel autoregressive models for multi-agent combinatorial optimization.
Advances in Neural Information Processing

Systems **38**, pp. 167130–167160.
Cited by: Related Work.

Bo-Hu et al. (2010)
L. Bo-Hu, Z. Lin, W. Shi-Long, T. Fei, C. Jun-Wei, J. Xiao-dan, S. Xiao, and C. Xu-dong
Cloud manufacturing: a new service-oriented networked manufacturing model.
Computer integrated manufacturing system **16** (01), pp. 0.
Cited by: Introduction.

Branke et al. (2015)
J. Branke, T. Hildebrandt, and B. Scholz-Reiter
Hyper-heuristic evolution of dispatching rules: a comparison of rule representations.
Evolutionary computation **23** (2), pp. 249–277.
Cited by: Introduction, Related Work.

Brucker et al. (1994)
P. Brucker, B. Jurisch, and B. Sievers
A branch and bound algorithm for the job-shop scheduling

problem.
Discrete applied mathematics **49** (1-3), pp. 107–127.
Cited by: Related Work.

Caldeira and Gnanavelbabu (2019)

R. H. Caldeira and A. Gnanavelbabu
Solving the flexible job shop scheduling problem using an improved jaya algorithm.
Computers & Industrial Engineering **137**, pp. 106064.
Cited by: Related Work.

Chaudhry and Khan (2016)

I. A. Chaudhry and A. A. Khan

A research survey: review of flexible job shop scheduling techniques.
International Transactions in Operational Research **23** (3), pp. 551–591.
Cited by: Introduction.

Chen and Matis (2013)

B. Chen and T. I. Matis
A flexible dispatching rule for minimizing tardiness in job shop scheduling.
International Journal of Production Economics **141** (1)

Production Economics **141** (1), pp. 360–365.
Cited by: Related Work.

Dat et al. (2025) T. T. Dat, P. A. Khoi, T. Q. Khai, D. D. Dong, and V. Van Khu *NeuFACO: neural focused ant colony optimization for traveling salesman problem*. In 2025 RIVF International Conference on Computing and Communication Technologies (RIVF), pp. 439–444.
Cited by: Related Work.

Demir and İşleyen (2013) Y. Demir and S. K. İşleyen Evaluation of mathematical models for flexible job-shop scheduling problems. *Applied Mathematical Modelling* **37** (3), pp. 977–988.
Cited by: Related Work, Related Work.

Gao et al. (2019) K. Gao, Z. Cao, L. Zhang, Z. Chen, Y. Han, and Q. Pan A review on swarm intelligence and evolutionary algorithms for solving flexible job shop

scheduling problems.
IEEE/CAA journal of automatica sinica **6** (4), pp. 904–916.
Cited by: Related Work.

Ghaedy-Heidary et al. (2024)

E. Ghaedy-Heidary, E. Nejati, A. Ghasemi, and S. A. Torabi
A simulation optimization framework to solve stochastic flexible job-shop scheduling problems—case: semiconductor manufacturing.
Computers & Operations Research **163**, pp. 106508.
Cited by: Introduction.

Han and Yang (2021)

B. Han and J. Yang
A deep reinforcement learning based solution for flexible job shop scheduling problem.
International Journal of Simulation Modelling **20** (2), pp. 375–386.
Cited by: Introduction.

Joshi et al. (2019)

C. K. Joshi, T. Laurent, and X. Bresson
An efficient graph convolutional network

convolutional network technique for the travelling salesman problem. *arXiv preprint arXiv:1906.01227*. Cited by: Related Work.

Kovacova and Lăzăroiu (2021)

M. Kovacova and G. Lăzăroiu Sustainable organizational performance, cyber-physical production networks, and deep learning-assisted smart process planning in industry 4.0-based manufacturing systems. *Economics, Management and Financial Markets* **16** (3), pp. 41–54. Cited by: Introduction.

Li et al. (2015)

S. Li, L. D. Xu, and S. Zhao The internet of things: a survey. *Information systems frontiers* **17** (2), pp. 243–259. Cited by: Introduction.

Liao et al. (2025)

Z. Liao, J. Chen, D. Wang, Z. Zhang, and J. Wang BOPO: neural combinatorial

optimization via best-anchored and objective-guided preference optimization.
arXiv preprint arXiv:2503.07580.
Cited by: Introduction.

Liu et al. (2015)

T. Liu, Y. Chen, and J. Chou
Solving distributed and flexible job-shop scheduling problems for a real-world fastener manufacturer.
IEEE Access **2**, pp. 1598–1606.
Cited by: Introduction.

Luttmann and Xie (2025)

L. Luttmann and L. Xie
Multi-action self-improvement for neural combinatorial optimization.
arXiv preprint arXiv:2510.12273.
Cited by: Related Work.

Mao et al. (2019)

S. Mao, B. Wang, Y. Tang, and F. Qian
Opportunities and challenges of artificial intelligence for green manufacturing in the process industry.

Engineering **5** (6), pp. 995–1002.
Cited by: Introduction.

Meng et al. (2020) — L. Meng, C. Zhang, Y. Ren, B. Zhang, and C. Lv
Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem.
Computers & industrial engineering **142**, pp. 106347.
Cited by: Related Work.

Müller et al. (2022) — D. Müller, M. G. Müller, D. Kress, and E. Pesch
An algorithm selection approach for the flexible job shop scheduling problem: choosing constraint programming solvers through machine learning.
European Journal of Operational Research **302** (3), pp. 874–891.
Cited by: Related Work.

Nouiri et al. (2018) — M. Nouiri, A. Bekrar, A. Jemai, S. Niar, and A. C. Ammari

S. Niar, and A. O. Ammari
An effective and distributed particle swarm optimization algorithm for flexible job-shop scheduling problem.
Journal of Intelligent Manufacturing **29** (3), pp. 603–615.
Cited by: Related Work.

Ren et al. (2017)
L. Ren, L. Zhang, L. Wang, F. Tao, and X. Chai
Cloud manufacturing: key characteristics and applications.
International journal of computer integrated manufacturing **30** (6), pp. 501–515.
Cited by: Introduction.

Schulman et al. (2015)
J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel
High-dimensional continuous control using generalized advantage estimation.
arXiv preprint arXiv:1506.02438.
Cited by: Training Algorithm.

Schulman et al. (2017)
J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov
Proximal policy optimization algorithms.
arXiv preprint arXiv:1707.06347.
Cited by: Training Algorithm.

Sels et al. (2012)
V. Sels, N. Gheysen, and M. Vanhoucke
A comparison of priority rules for the job shop scheduling problem under different flow time-and tardiness-related objective functions.
International Journal of Production Research **50** (15), pp. 4255–4270.
Cited by: Related Work.

Smit et al. (2025)
I. G. Smit, Y. Wu, P. Troubil, Y. Zhang, and W. P. Nuijten
Neural combinatorial optimization for stochastic flexible job shop scheduling problems.
In Proceedings of the AAAI

Proceedings of the AAAI Conference on Artificial Intelligence, Vol. **39**, pp. 26678–26687. Cited by: Introduction.

Song et al. (2022) — W. Song, X. Chen, Q. Li, and Z. Cao. Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics* **19** (2), pp. 1600–1610. Cited by: Introduction, Experimental Settings.

Tutumlu and Saraç (2023) — B. Tutumlu and T. Saraç. A mip model and a hybrid genetic algorithm for flexible job-shop scheduling problem with job-splitting. *Computers & Operations Research* **155**, pp. 106222. Cited by: Introduction.

Wang et al. (2023) — R. Wang, G. Wang, J. Sun, F. Deng, and J. Chen. Flexible job shop scheduling via dual attention network

via dual attention network-based reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems* **35** (3), pp. 3091–3102.
Cited by: Introduction, Experimental Settings.

Wang et al. (2025) — Y. Wang, T. Guo, and Z. Liu
An expectation-maximization algorithm-based autoregressive model for the fuzzy job shop scheduling problem.
arXiv preprint arXiv:2502.00018.
Cited by: Introduction.

Xiao et al. (2026) — X. Xiao, C. Zhang, W. Song, and Z. Cao
Resched: rethinking flexible job shop scheduling from a transformer-based architecture with simplified states.
arXiv preprint arXiv:2603.07020.
Cited by: Introduction, Related Work, Experimental Settings.

Xiao et al. (2024a)

Y. Xiao, D. Wang, B. Li, H. Chen, W. Pang, X. Wu, H. Li, D. Xu, Y. Liang, and Y. Zhou
Reinforcement learning-based nonautoregressive solver for traveling salesman problems.
IEEE Transactions on Neural Networks and Learning Systems **36** (7), pp. 13402–13416.
Cited by: Related Work.

Xiao et al. (2024b)

Y. Xiao, D. Wang, B. Li, M. Wang, X. Wu, C. Zhou, and Y. Zhou
Distilling autoregressive models to obtain high-performance non-autoregressive solvers for vehicle routing problems with faster inference speed.
In Proceedings of the AAAI conference on artificial intelligence,
Vol. **38**, pp. 20274–20283.
Cited by: Related Work.

Xiao et al. (2025)

Y. Xiao, D. Wang, X. Wu, Y. Wu, B. Li, W. Du, L. Wang, and Y.

Zhou
Improving generalization of neural vehicle routing problem solvers through the lens of model architecture.
Neural Networks **187**, pp. 107380.
Cited by: Introduction.

Xie et al. (2019)
J. Xie, L. Gao, K. Peng, X. Li, and H. Li
Review on flexible job shop scheduling.
IET collaborative intelligent manufacturing **1** (3), pp. 67–77.
Cited by: Introduction.

Xu et al. (2024)
S. Xu, Y. Li, and Q. Li
A deep reinforcement learning method based on a transformer model for the flexible job shop scheduling problem.
Electronics **13** (18), pp. 3696.
Cited by: Introduction, Related Work.

Xu et al. (2025)
Y. Xu, M. Zhang, D. Wang, M. Yang, and C. Liang

Yang, and C. Liang
Hybrid gaussian quantum particle swarm optimization and adaptive genetic algorithm for flexible job-shop scheduling problem.
Engineering Applications of Artificial Intelligence **154**, pp. 110882.
Cited by: Introduction, Introduction.

Yuan et al. (2024)
E. Yuan, L. Wang, S. Cheng, S. Song, W. Fan, and Y. Li
Solving flexible job shop scheduling problems via deep reinforcement learning.
Expert Systems with Applications **245**, pp. 123019.
Cited by: Introduction.

Zeng et al. (2022)
Z. Zeng, X. Li, and C. Bai
A deep reinforcement learning approach to flexible job shop scheduling.
In 2022 IEEE international conference on systems, man, and cybernetics (SMC), pp. 884–890.
Cited by: Related Work.

Zhang et al. (2009)
G. Zhang, X. Shao, P. Li, and L. Gao
An effective hybrid particle swarm optimization algorithm for multi-objective flexible job-shop scheduling problem.
Computers & industrial engineering **56** (4), pp. 1309–1318.
Cited by: Related Work.

Zhang et al. (2023)
J. Zhang, Z. He, W. Chan, and C. Chow
DeepMAG: deep reinforcement learning with multi-agent graphs for flexible job shop scheduling.
Knowledge-Based Systems **259**, pp. 110083.
Cited by: Introduction, Related Work.

Zhang et al. (2025)
J. Zhang, Y. Jiang, B. Guo, T. Liu, D. Hu, J. Zhang, Y. Deng, H. Wang, J. Yang, and X. Ding
Dynamic scheduling for cloud manufacturing with uncertain events by hierarchical reinforcement learning and

reinforcement learning and attention mechanism. *Knowledge-Based Systems* **316**, pp. 113335.
Cited by: Introduction.

Zhao and Deng (2024)
C. Zhao and N. Deng
An actor-critic framework based on deep reinforcement learning for addressing flexible job shop scheduling problems. *Math. Biosci. Eng* **21** (1), pp. 1445–1471.
Cited by: Introduction.

Zhao et al. (2025)
P. Zhao, Y. Zhou, D. Wang, Z. Cao, Y. Xiao, X. Wu, Y. Li, H. Liu, W. Du, Y. Jiang, et al. *Dual operation aggregation graph neural networks for solving flexible job-shop scheduling problem with reinforcement learning*. In Proceedings of the ACM on Web Conference 2025, pp. 4089–4100.
Cited by: Introduction, Training Algorithm, Experimental Settings.

Zhou et al. (2019) Y. Zhou, J. Yang, and L. Zheng Multi-agent based hyper-heuristics for multi-objective flexible job shop scheduling: a case study in an aero-engine blade manufacturing plant. *Ieee Access* **7**, pp. 21147–21176.
Cited by: Introduction.

Experimental support, please view the build logs for errors. Generated by LaTeXML (https://math.nist.gov/~BMiller/LaTeXML/).

Instructions for reporting errors

We are continuing to improve HTML versions of papers, and your feedback helps enhance accessibility and mobile support. To report errors in the HTML that will help us improve conversion and rendering, choose any of the methods listed below:

- Click the "Report Issue" (🐞) button, located in the page header.

Tip: You can select the relevant text first, to include it in your report.

Our team has already identified the following issues (https://github.com/arXiv/html_feedback/issues). We appreciate your time reviewing and reporting rendering errors we may not have found yet. Your efforts will help us improve the HTML versions for all readers, because disability should not be a barrier to accessing research.

Thank you for your continued support in championing open access for all.

Have a free development cycle? Help support accessibility at arXiv! Our collaborators at LaTeXML maintain a list of packages that need conversion (https://github.com/brucemiller/LaTeXML/wiki/Porting-LaTeX-packages-for-LaTeXML), and welcome developer contributions (https://github.com/brucemiller/LaTeXML/issues).